UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

Chapitre 5 :

Les Tableaux à deux dimensions

Les tableaux dynamiques

Imaginons que l'on veux saisir des notes pour un calcul de moyenne, mais on ne connait pas d'avance le nombre de notes à saisir

- On pourrait déclarer un tableau avec 100 valeurs ..
- Mais on n'est pas toujours sûr que ce serait suffisant et ça prend beaucoup de place pour rien si on a besoin de moins

utilisation d'une taille maximale

```
Tableau Notes[500] : Réel
Variables nb, i : Entier
Début
Ecrire("Combien y a-t-il de notes à saisir ? ")
Lire(nb)
Ecrire("saisir les notes ")
Pour i=0 à nb
    Lire(Notes[i])
FinPour
…
Fin
```

Les tableaux dynamiques

- L'utilisation d'une taille maximale peut avoir comme problèmes:
  - ✓Insuffisance de l'espace mémoire réservé
  - ✓Gaspillage d'espace mémoire (des cases réservées sans être utilisées)
- Les tableaux dynamiques sont utilisés lorsqu'on ne connaît pas à l'avance la taille :

```
Tableau Notes[] : Réel
Variables nb : Entier
Début
Ecrire("Combien y a-t-il de notes à saisir ? ")
Lire(nb)
Redim Notes[nb]
…
Fin
```

déclaration d'un tableau dynamique

redimensionnement de la taille du tableau Notes à nb éléments

Les tableaux à deux dimensions

Les langages de programmation permettent de déclarer des tableaux à deux dimensions. Ceci est utile pour, par exemple, représenter des matrices

Déclaration:

Tableau identificateur**[dim1][dim2] : type**

- **identificateur :** désigne le nom du tableau
- **dim1 :** désigne le nombre de lignes du tableau
- **dim2 :** désigne le nombre de colonnes du tableau
- **type :** représente le type des éléments du tableau

En python : identificateur = [[] * **dim2** for _ in range(**dim1**)]

Les tableaux à deux dimensions

Tableau à deux dimensions

```
MAT = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]
 ]
```

Algorithme

```
Tableau MAT[][]: Entier
Début
Ecrire("Donnez le nombre de lignes et de colonnes")
Lire(N,M)
Redim MAT[N][M]
```

Code Python

```
# Tableau à deux dimensions vide de taille 3x3, initialisé avec des zéros
N = 3  # Nombre de lignes
M = 3  # Nombre de colonnes
MAT = [[0] * M for _ in range(N)]
```

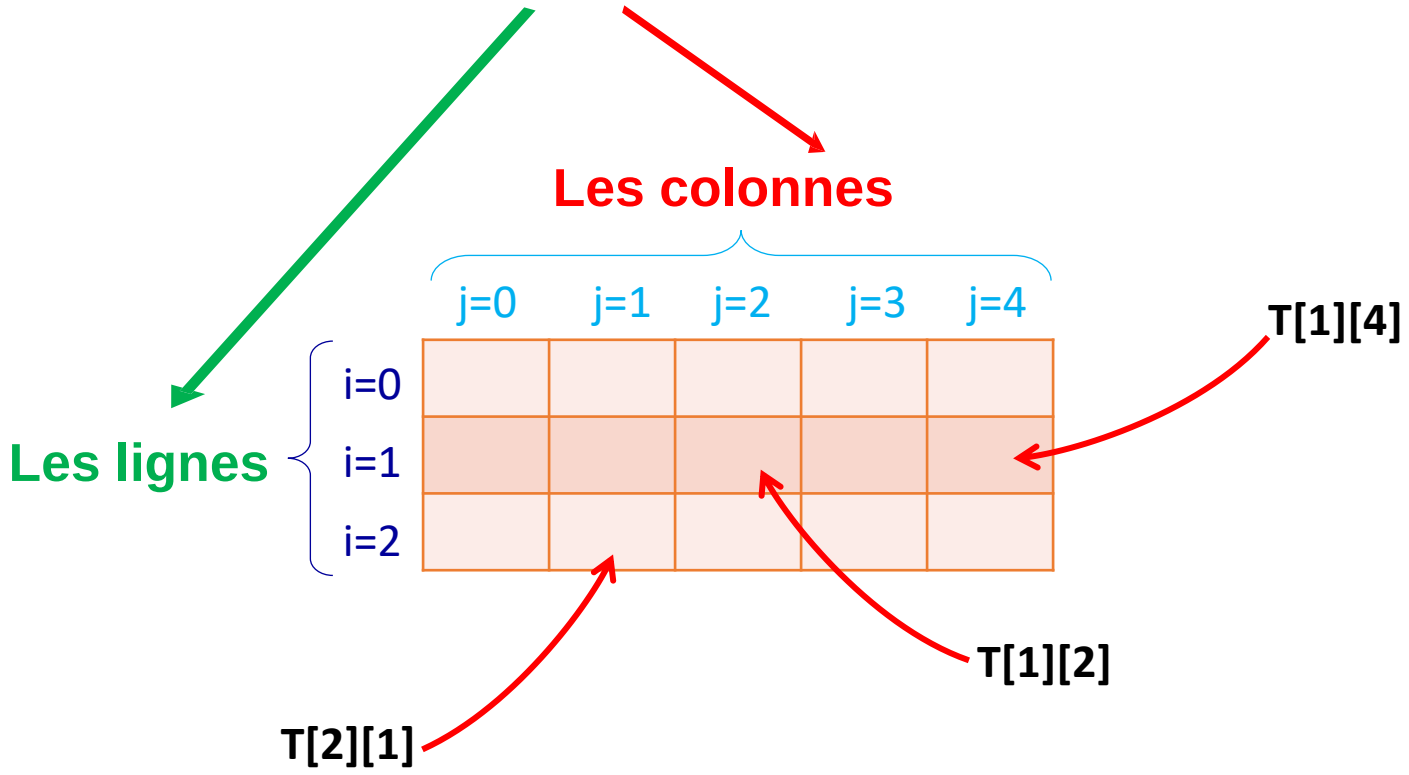
Les tableaux à deux dimensions

- Exemple: **Tableau T[3][2]: Entier**
  - ✓ T est un tableau d'entiers à deux dimensions.
  - ✓ La première dimension a une taille égale à 3
  - ✓ La deuxième a une taille égale à 2
  - ✓ T peut être vue comme une matrice à 3 lignes

- Soit T[n][m] un tableau d'entiers:
  - ✓ T contient n * m cases.
  - ✓ Exemple : Dans l'exemple précédent, T contient 6 cases.

Les tableaux à deux dimensions

Exemple: Tableau T[3][5] : Entier



Les tableaux à deux dimensions

- Une matrice peut être considérée comme le **regroupement** de plusieurs tableaux de même taille :
  - ✓ Regrouper trois tableaux ayant chacun 5 éléments aboutira à une matrice de 3 lignes (tableaux) et 5 colonnes (Cases). (Voir Exemple précédent)

- **Indice** : les lignes d'une matrice sont numérotées par des entiers successifs de même que les cases de chaque ligne. Ces numéros sont appelés indices de ligne et de colonne. En programmation, il dépend du langage utilisé que la première case porte le numéro 0 ou 1 (ou autre) ; la case suivante porte le numéro suivant.
  - ✓ Nous respecterons la consigne donnée pour les tableaux unidimensionnels de débuter par "0"

Les tableaux à deux dimensions

Pour accéder (en lecture ou en écriture) à l'élément de la matrice se trouvant à l'intersection de **la ligne i** et **la colonne j**, on utilise la syntaxe suivante :

Nom_tableau [i][j]

Exemple : **Tableau T[3][2] : Réel**

- L'instruction : **T[2][1] ← -6** met la valeur -6 dans la case se trouvant dans la 3ème ligne et la 2ème colonne.
- L'instruction : **Lire(T[2][1])** permet de lire une valeur au clavier et de la mettre dans la case se trouvant dans la ligne d'indice 2 et la colonne d'indice 1.

Les tableaux à deux dimensions

- Le traitement de plusieurs cases de plusieurs lignes d'un tableau à deux dimensions se fait en général par des **boucles imbriquées**.
- **Exemple** :
  - Si nous avons à remplir les cases de la matrice A ci-dessus nous aurons à utiliser les boucles suivantes:

```
Pour I= 0 à 2
        Pour J=0 à 6
                Lire (A [I][J])
        FinPour
FinPour
```

  - A l'exécution : l'indice I serait incrémenté après que l'instruction de lecture aurait été exécutée 7 fois (l'indice J aurait changé en o à 6). A chaque nouvelle valeur, l'indice I bloquera pour permettre à l'indice J de faire un tour complet.

Les tableaux à deux dimensions

Exemple: Saisie d'une matrice

```
Algorithme Saisie_matrice
Variables i,j,n,m,L,C : Entier
Tableau A[][]: Réel
Début
Ecrire("Donnez le nombre de lignes et de colonnes")
Lire(L,C)
Redim A[L][C]
Pour i=0 à L-1
    Ecrire("saisie de la ligne N°", i+1)
    Pour j=0 à C-1
        Ecrire("Entrez l'élément [" , i+1 , "][" , j+1 , "]")
        Lire(A[i][j])
    FinPour
FinPour
Fin
```

Les tableaux à deux dimensions

```
Algorithme Saisie_matrice
Variables i,j,L,C : Entier
Tableau A[][]: Réel
Début
Ecrire("Donnez le nombre de lignes et de colonnes")
Lire(L,C)
Redim A[L][C]
Pour i=0 à L-1
    Ecrire("saisie de la ligne N°", i+1)
    Pour j=0 à C-1
        Ecrire("Entrez l'élément [" , i+1 , "] [ " ,j+1 , "]")
        Lire(A[i][j])
    FinPour
FinPour
Fin
```

```
L = 0
C = 0
i = 0
j = 0
print("Donnez le nombre de lignes et de colonnes")
L = int(input("Nombre de lignes : "))
C = int(input("Nombre de colonnes : "))
A = [[0.0] * C for _ in range(L)]
for i in range(L):
    print(f"Saisie de la ligne N° {i+1}")
    for j in range(C):
        A[i][j] = float(input(f"Entrez l'élément [{i+1}] [{j+1}] : "))

print("Matrice saisie :")
for i in range(L):
    for j in range(C):
        print(A[i][j], end=" ")
    print()
```

Les tableaux à deux dimensions

Exemple: Affichage d'une matrice

On considère une matrice **T** de **L** lignes et **C** colonnes :

Affichage de la ligne i

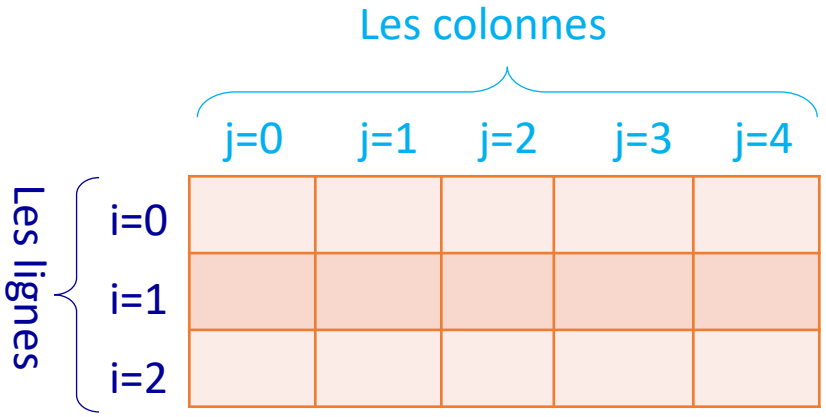
```
Pour j=0 à C-1
    Ecrire(T[i][j], " ")
FinPour
```

Affichage de la colonne j

```
Pour i=0 à L-1
    Ecrire(T[i][j], " ")
FinPour
```

Affichage de la matrice

```
Pour i=0 à L-1
    Pour j=0 à C-1
        Ecrire(T[i][j], " ")
    FinPour
    Ecrire("\n")
FinPour
```



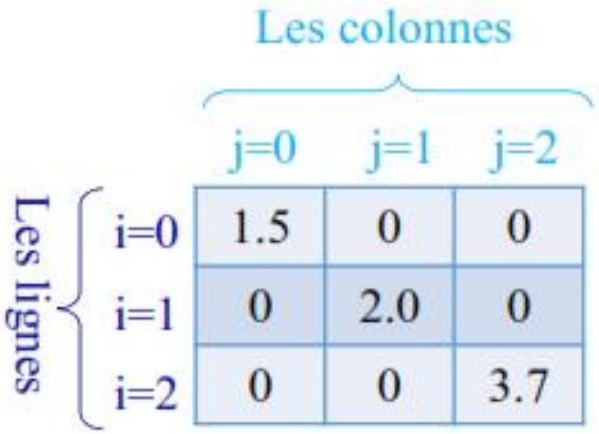
Les tableaux à deux dimensions

Affichage d'une matrice : Code Python

```
# Déclaration et initialisation d'un tableau à deux dimensions
T = [[0.0, 0.0,0.0], [0.0, 0.0,0.0], [0.0, 0.0,0.0]]

# Accès et modification des éléments du tableau
T[0][0] = 1.5
T[1][1] = 2.0
T[2][2] = 3.7

# Affichage du tableau
for i in range(3):
    for j in range(3):
        print(T[i][j], end=" ")
    print(« /N»)
```



Exercices

1. Que produit l'exécution de ce programme?

```
Tableau   X[2][3]  :  Entier
Variables i , j , val : Entier
Début
val ← 3
Pour i=0 à 1
    Pour j=0 à 2
        X[i][j] ← val
        val ← val+2
    FinPour
FinPour
Pour i=0 à 1
    Pour j=0 à 2
        Ecrire(X[i][j], "  " )
    FinPour
    Ecrire(" \n ")
FinPour
Fin
```

3 5 7

9 11 13

Exercices

2. Que produit l'exécution de ce programme?

```
Tableau X[4][2] : Entier
Variables k , m : Entier
Début
Pour k=0  à  3
    Pour m=0  à  1
    X[k][m] ← k+m
    FinPour
FinPour
Pour k=0  à  3
    Pour m=0  à  1
       Ecrire(X[k][m], "  ")
    FinPour
    Ecrire(" \n ")
FinPour
Fin
```

0 1
1 2
2 3
3 4

Exercices

1. Insertion d'un élément dans un tableau

Soit **T** un tableau trié de N éléments. Ecrire un algorithme qui permet d'insérer un élément **x** dans le tableau **T** en gardant l'ordre dans le tableau **T**

Exemple:

x=30

12	20	28	34	44	60	72

12	20	28	30	34	44	60	72

2. Suppression d'un élément à partir d'un tableau

Soit **T** un tableau de N éléments. Ecrire un algorithme qui permet de supprimer un élément **x** du tableau **T**

Exemple:

x=3

12	28	3	14	35	3	60	72

12	28	14	35	60	72

Solution

1. Insertion d'un élément dans un tableau

12	20	28	34	44	60	72	
12	20	28	30	34	44	60	72

```
Tableau T[] : Entier
Variables nb,i,pos,x : Entier
Début
Ecrire("saisir le nombre de valeurs")
Lire(nb)
Redim T[nb]
Ecrire("saisir les valeurs")
Pour i=0 à nb-1
  Lire(T[i])
FinPour
Ecrire("saisir la valeur à insérer")
Lire(x)
Redim T[nb+1]
i ← 0
TantQue (i < nb  et T[i]<x)
  i ← i+1
FinTanQue
pos ← i

Pour i=nb à pos+1 Pas de -1
  T[i] ← T[i-1]
FinPour

T[pos] ← x
Fin
```

Recherche de la position

Décalage des cases entre pos et nb

Insertion de x

Solution

1. Insertion d'un élément dans un tableau

```
nb = 0
i = 0
pos = 0
x = 0.0

nb = int(input("Saisir le nombre de valeurs :  "))
T = [0.0] * nb

print("Saisir les valeurs : ")
for i in range(nb):
    T[i] = float(input())

x = float(input("Saisir la valeur à insérer : "))

T.append(0.0)
```

12	20	28	34	44	60	72

12	20	28	30	34	44	60	72

```
i = 0

while i < nb and T[i] < x:
    i += 1

pos = i

for i in range(nb, pos, -1):
    T[i] = T[i-1]

T[pos] = x
```

Recherche de la position

Décalage des cases entre pos et nb

Insertion de x

Solution

1. Suppression d'un élément dans un tableau

12	28	3	14	35	3	60	72

12	28	14	35	60	72

```
Tableau T[] : Réel
Variables nb,i,pos,x : Entier
Début
Ecrire("saisir le nombre de valeurs")
Lire(nb)
Redim T[nb]
Ecrire("saisir les valeurs")
Pour i=0 à nb-1
  Lire(T[i])
FinPour
Ecrire("saisir la valeur à supprimer")
Lire(x)
i ← 0
TantQue (T[i]<>x)
  i ← i+1
FinTanQue
pos ← i

Pour i= pos à nb-1 Pas de 1
T[i] ← T[i+1]
FinPour
Redim T[nb-1]
Fin
```

Recherche de la position

suppression

Solution

1. Suppression d'un élément dans un tableau

12	28	3	14	35	3	60	72

12	28	14	35	60	72

```
nb = 0
i = 0
pos = 0
x = 0.0
nb = int(input("Saisir le nombre de valeurs : "))

T = [0.0] * nb

print("Saisir les valeurs : ")
for i in range(nb):
    T[i] = float(input())

x = float(input("Saisir la valeur à supprimer : "))
```

```
i = 0
while i < nb and T[i] != x:
    i += 1

pos = i
```

Recherche de la position

```
for i in range(pos, nb-1):
    T[i] = T[i+1]

T = T[:-1]
```

Suppression

Exercices

Exercice :Transfert d'une matrice vers un tableau

Faire le programme qui transfère les éléments d'une matrice MAT (N lignes et M colonnes dimensions maximales: 10 lignes et 10 colonnes) dans un tableau T à une dimension de taille N*M.

MAT =

1	2	3	4
5	6	7	8
9	10	11	12

Après transfert on aura :

T =

1	2	3	4	5	6	7	8	9	10	11	12

Solution

Algorithme

```
Tableau MAT[N][M] : Entier
Tableau T[N*M] : Entier
Variables i, j, k : Entier
Début
k ← 0
Pour i = 0 à N – 1
   Pour j = 0 à M – 1
      T[k] ← MAT[i][j]
      k ← k + 1
   FinPour
FinPour
Pour k = 0 à (N * M) – 1
    Ecrire(T[k], " ")
FinPour
Fin
```

Code Python

```
N = 3
M = 4
MAT = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10,
11, 12] ]
T = [0] * (N * M)
k = 0
for i in range(N):
   for j in range(M):
            T[k] = MAT[i][j]
            k += 1
for element in T:
    print(element, end=" ")
```